

Plano de Testes: Sebo Comunitário

Projeto: Sebo | Responsável: Daniela (QA)

Documentos de referência:

- Acordo de Manutenibilidade e Engenharia de Software
- Documento de Especificação de Requisitos e DoR

1. Objetivo

Definir a estratégia, o ambiente e os casos de teste necessários para garantir que o sistema, na sua versão N1 (cadastro de usuários, acervo de livros, sistema de créditos e solicitação de troca), atenda às regras de negócio especificadas, com atenção especial à RN Crítica: um usuário só pode resgatar um livro se tiver pelo menos 1 crédito ativo na conta.

2. Escopo

Dentro do escopo (N1)

RF	Funcionalidade
RF01	Gestão de Contas (cadastro, login)
RF02	Cadastro de Doação de Livro
RF03	Carteira de Créditos
RF04	Resgate de Livros
RF05	Catálogo Público

Requisitos Não Funcionais cobertos

RNF	Categoria
RNF01	Confiabilidade / Atomicidade das transações
RNF02	Segurança / Autenticação
RNF03	Segurança / Criptografia de senha
RNF05	Desempenho do catálogo (< 500ms)

3. Estratégia de Testes

Conforme definido no Acordo de Manutenibilidade (Seção 4):

- Foco primário: regras de negócio no backend/domain, com destaque para a lógica de créditos (doação e resgate).
- Frequência: antes de cada entrega oficial, será executado o roteiro completo cobrindo o fluxo principal e as regras de negócio críticas.
- Níveis de teste: funcionais manuais/exploratórios (cadastro, doação, resgate); testes de API via Postman/Insomnia para validar regras server-side; testes de concorrência para a RN Crítica (RNF01).

Definition of Done: validação do QA

Nenhuma tarefa é considerada concluída sem:

- Validação pelo roteiro de testes deste plano, sem erros críticos.
- Merge na branch main/develop.

4. Ambiente de Testes

- Ferramenta de API: Postman ou Insomnia, com coleção dedicada às rotas do backend (POST /usuarios, POST /doacao, POST /resgate, GET /livros).
- Pré-requisito: rotas do Leonardo (backend) disponíveis, ainda que como mock inicial.
- Banco de dados: PostgreSQL, validação direta das tabelas Usuarios, Livros e Transacoes_Credito quando necessário.
- Ponto de sincronização: reunião 'Daniela + Devs', rodando as coleções do Postman contra a API do Leonardo para validar a RN Crítica.

5. Matriz de Casos de Teste

RF01: Gestão de Contas (Cadastro de Usuário)

ID	Cenário	Pré-condição	Passos	Resultado Esperado	Prior.
TC01	Cadastro com dados válidos	Nenhuma	Preencher nome, e-mail e senha válidos e confirmar	Conta criada com sucesso; saldo_creditos = 0	Alta
TC02	Cadastro com e-mail já existente	Já existe usuário com o e-mail X	Tentar cadastrar novamente com o mesmo e-mail	Sistema exibe: "E-mail já cadastrado. Por favor, faça login." Conta não é duplicada	Alta
TC03	Cadastro com campos obrigatórios vazios	Nenhuma	Enviar formulário sem preencher nome/e-mail/senha	Sistema bloqueia o envio e sinaliza os campos obrigatórios	Média
TC04	Senha armazenada com segurança (RNF03)	Usuário cadastrado	Inspecionar coluna senha_hash no banco	Senha salva com hash (BCrypt), nunca em texto puro	Alta

RF02 / RF03: Doação de Livro e Crédito

ID	Cenário	Pré-condição	Passos	Resultado Esperado	Prior.
TC05	Doação com dados completos (cenário feliz)	Usuário logado	Preencher título, autor e gênero; confirmar doação	Livro salvo com status DISPONIVEL; saldo +1; mensagem: "Livro cadastrado! Você ganhou 1 crédito."	Alta
TC06	Doação com campo obrigatório faltando	Usuário logado	Enviar formulário sem título/autor/gênero	Sistema bloqueia a ação e destaca os campos em vermelho; nenhum crédito é gerado	Média
TC07	Registro da transação de entrada	Usuário logado, doação concluída	Verificar tabela Transacoes_Credito após doação	Registro criado com tipo: ENTRADA, descricao: 'Doação de livro'	Alta
TC08	Proporção estrita 1:1	Usuário logado	Doar 3 livros em sequência	Saldo aumenta exatamente 3 (nunca mais, nunca menos)	Alta

RF04: Resgate de Livros (RN CRÍTICA)

ID	Cenário	Pré-condição	Passos	Resultado Esperado	Prior.
TC09	Resgate com saldo suficiente (cenário feliz)	Usuário com saldo ≥ 1 , livro DISPONIVEL	Acessar livro e clicar em "Resgatar (1 Crédito)"	Saldo debitado em -1; livro muda para RESGATADO e é vinculado ao usuário; mensagem de confirmação	Alta
TC10	Resgate com saldo = 0 (bloqueio frontend)	Usuário com saldo = 0	Clicar em "Resgatar"	Sistema bloqueia no frontend; exibe: "Você precisa de pelo menos 1 crédito ativo para resgatar um livro. Doe um livro para ganhar créditos."	Crítica
TC11	Resgate com saldo = 0 via chamada direta à API (bypass do frontend)	Usuário com saldo = 0	Enviar requisição POST /resgate diretamente (Postman/Insomnia)	Backend também bloqueia (validação server-side); mesma mensagem de erro; nenhuma alteração no banco	Crítica
TC12	Registro da transação de saída	Resgate concluído	Verificar tabela Transacoes_Credito	Registro criado com tipo: SAIDA, descricao: 'Resgate de livro'	Alta
TC13	Tentativa de resgatar livro já resgatado	Livro com status RESGATADO	Outro usuário tenta resgatar o mesmo livro	Sistema impede o resgate; livro não aparece mais como disponível	Alta
TC14	Condição de corrida: 2 resgates simultâneos, saldo = 1 (RNF01)	Usuário com saldo = 1, dois livros disponíveis	Disparar duas requisições de resgate ao mesmo tempo	Apenas uma transação é confirmada; saldo final = 0 (nunca negativo); transação atômica no PostgreSQL	Crítica

RF05: Catálogo Público

ID	Cenário	Pré-condição	Passos	Resultado Esperado	Prior.
TC15	Listagem exibe apenas livros disponíveis	Existem livros DISPONIVEL e RESGATADO	Acessar catálogo	Somente livros com status DISPONIVEL aparecem na listagem pública	Média
TC16	Busca/filtro por título, autor ou gênero	Catálogo com múltiplos livros	Buscar por termo específico	Resultados filtrados corretamente	Média
TC17	Tempo de resposta do catálogo (RNF05)	Catálogo populado	Medir tempo de resposta da requisição de listagem	Retorno em menos de 500ms	Baixa

RNF02: Segurança / Autenticação

ID	Cenário	Pré-condição	Passos	Resultado Esperado	Prior.
TC18	Doação/resgate sem token de sessão	Visitante não autenticado	Tentar POST /doacao ou POST /resgate sem token	Requisição bloqueada (401/403)	Crítica
TC19	Token inválido ou expirado	Usuário com token expirado	Tentar doar ou resgatar	Requisição bloqueada; sessão deve ser renovada/login exigido	Alta

Legenda de Prioridade

- Crítica: bloqueia o release se falhar (RN Crítica ou segurança).
- Alta: deve ser corrigido antes da entrega, mas não bloqueia sozinho.
- Média/Baixa: registrado como bug, priorizado conforme severidade e capacidade do sprint.

Observação de testabilidade

TC10, TC11 e TC14 validam a RN Crítica sob três ângulos diferentes (frontend, backend isolado, concorrência). Nenhum substitui o outro. Um bug de saldo negativo tipicamente só aparece no TC14, mesmo que TC10 e TC11 passem.

6. Registro e Resolução de Bugs

Conforme o Acordo de Manutenibilidade (Seção 4.2), todo bug identificado deve ser documentado com:

- Passos para reproduzir
- Resultado esperado vs. Resultado obtido
- Severidade (Crítica / Alta / Média / Baixa)

Regra de bloqueio: bugs de alta severidade (e todos os classificados como Crítica nesta matriz) bloqueiam o encerramento da tarefa e impedem o merge/release até serem corrigidos.

7. Critério de Saída (Exit Criteria)

A versão N1 é considerada pronta para entrega quando:

- 100% dos casos de prioridade Crítica (TC10, TC11, TC14, TC18) passarem.
- Nenhum bug de severidade Alta ou Crítica estiver em aberto.
- Os casos de prioridade Alta tiverem taxa de sucesso $\geq 90\%$.